📚 AI-Powered Document Question Answering System Using RAG

A production-ready **Retrieval-Augmented Generation (RAG)** web app that lets you upload one or multiple PDF documents and ask natural-language questions. Answers are generated **only from retrieved context** and shown with source citations (filename + page).

Built for **B.Tech Major Project** — simple, explainable, and deployable to Streamlit Community Cloud.

📌 Problem Statement

Large Language Models hallucinate when asked about private or recent documents. Users need a system that:

1. Understands the content of their own PDFs
2. Retrieves only the most relevant passages
3. Generates answers grounded in those passages with provenance (source + page)

🎯 Objectives

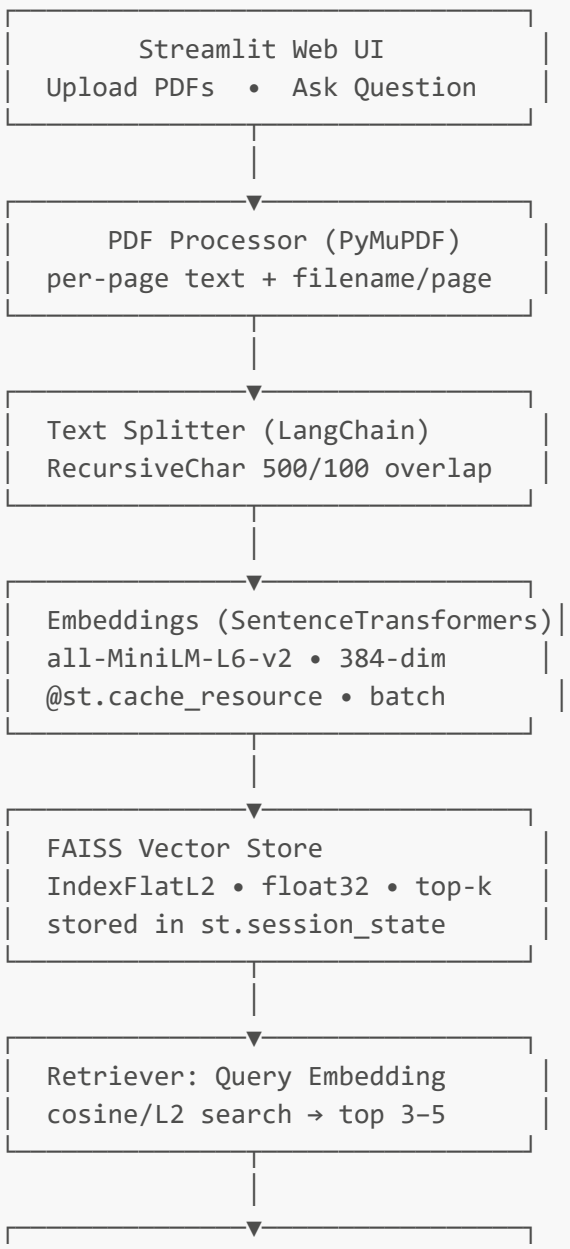
- Upload single or multiple PDFs via a clean web UI
- Extract text per page and preserve metadata (filename, page number)
- Chunk text intelligently, embed with SentenceTransformers, store in FAISS
- Retrieve top-k relevant chunks per question
- Generate answers with Groq LLM, strictly from context (no invention)
- Show citations, chat history, and handle all error cases gracefully
- Optimize for performance (cached models, no rebuild per question)

✨ Features

Feature	Details
PDF Upload	Single + multiple PDFs, shows filenames, validates .pdf only
Text Extraction	PyMuPDF per page, detects scanned/image PDFs gracefully
Chunking	`RecursiveCharacterTextSplitter` (500 chars, 100 overlap), keeps filename + page
Embeddings	`SentenceTransformer("all-MiniLM-L6-v2")` — cached with `@st.cache_resource`
Vector DB	FAISS `IndexFlatL2` (float32), batch embeddings, no rebuild per question
RAG QA	Query → embed → FAISS top-k (3–5) → prompt with context → Groq → answer

Feature	Details
Strict Prompt	“If answer not in context, say not available. Do not invent.”
Sources	Deduped badges: `filename.pdf | Page N`
Chat History	Session-based Q/A with Clear Chat button
Performance	Model loads once, PDFs processed only when files change, progress spinners
Error Handling	Empty/scanned PDFs, no text, empty question, missing/invalid API key, model unavailable, FAISS errors, too few chunks
Security	`.env` for `GROQ_API_KEY` / `GROQ_MODEL`, `.gitignore` prevents commit

System Architecture



```
                  |  Groq LLM (configurable model)   |
                  |  Context + Question → Answer      |
                  |  + Sources (filename | page)     |
                  └──────────────────────────────────┘


  Flow:

  PDF → Text Extraction → Text Chunking → Embedding Generation → FAISS Vector DB
  → Question Embedding → Similarity Search → Relevant Context → Groq LLM → Answer
  + Sources

  **Required RAG pipeline (exact):**
```

PDF Upload
↓
Text Extraction
↓
Text Chunking
↓
Embedding Generation
↓
FAISS Vector Database
↓
Similarity Search
↓
Relevant Context
↓
Groq LLM
↓
Answer + Sources

Technologies Used

- **Python 3.12**
- **Streamlit** — web UI
- **PyMuPDF (fitz)** — PDF text extraction per page
- **LangChain / langchain-text-splitters** — `RecursiveCharacterTextSplitter`
- **Sentence Transformers** — `all-MiniLM-L6-v2` (384-dim)
- **FAISS (faiss-cpu)** — vector similarity search (`IndexFlatL2`)
- **Groq API** — LLM (configurable, e.g., `openai/gpt-oss-120b`)
- **python-dotenv** — environment variables
- **torch, numpy** — embedding runtime

Project Structure

```
open_code/  (project root — QA_RAG)
│
├── app.py                        # Main Streamlit app (RAG UI + logic)
├── app_flask_backup.py           # Backup of original Flask implementation
(preserved)
├── requirements.txt              # Pinned compatible dependencies (Python 3.12)
├── .env.example                  # Placeholder for GROQ_API_KEY + GROQ_MODEL
├── .gitignore                    # Excludes .env, __pycache__, venv, faiss_store
├── README.md                     # This file
├── TESTING.md                    # Manual test cases & expected results
│
├── utils/                        # Modular RAG components
│   ├── __init__.py
│   ├── pdf_processor.py          # PyMuPDF + LangChain chunking
│   ├── embeddings.py             # Cached SentenceTransformer + batch embed
│   ├── vector_store.py           # FAISS IndexFlatL2 build + search
│   └── rag_pipeline.py           # Retrieval + Groq generation + sources
│
├── sample_docs/                  # Example text files for testing (optional)
├── uploads/                      # Temp upload dir (gitignored)
├── faiss_store/                  # Persisted index (gitignored)
└── venv/                         # Local virtual environment (gitignored)
```

File Descriptions

- **`app.py`** — Streamlit entry point. Handles upload, hashing to avoid reprocessing, calls utils, stores `chunks`/`faiss_index` in `st.session_state`, renders question input, calls `answer_question()`, shows answer + deduplicated sources + chat history + clear button, sidebar stats.
- **`utils/pdf_processor.py`** — `extract_text_from_pdf()` (fitz, per page), `chunk_pages()` (RecursiveCharacterTextSplitter), `process_uploaded_pdfs()` (validate multi-PDF, handle empty/scanned).
- **`utils/embeddings.py`** — `load_embedding_model()` cached with `@st.cache_resource`, `embed_texts()` batch float32, `embed_query()`.
- **`utils/vector_store.py`** — `build_faiss_index()` (IndexFlatL2, float32, contiguous), `search_faiss()` (clamps top_k, L2→similarity), `get_index_stats()`.
- **`utils/rag_pipeline.py`** — `get_groq_client()` cached, `get_groq_model()` env+default, `build_context()`, `deduplicate_sources()`, `generate_answer()` (strict prompt, model-unavailable handling, fallback extractive), `answer_question()` (full flow).
- **`requirements.txt`** — Verified compatible versions for Python 3.12 (Streamlit, PyMuPDF, LangChain, Groq, FAISS, SentenceTransformers, torch, torchvision, numpy).
- **`.env.example`** — Template with `GROQ_API_KEY=your_...` and `GROQ_MODEL=openai/gpt-oss-120b` (verified 2026-08-29).
- **`.gitignore`** — Prevents committing `.env`, caches, venv, stores.

🚀 Installation Instructions

Prerequisites

- Python **3.12** (check: `python --version`)
- A Groq API key (free at https://console.groq.com/keys)
- Windows / macOS / Linux

1) Clone or download

```
git clone <your-repo-url>
cd open_code
```

2) Create virtual environment

```
# Windows
python -m venv venv
venv\Scripts\activate

# macOS/Linux
python3 -m venv venv
source venv/bin/activate
```

3) Install dependencies

```
pip install --upgrade pip
pip install -r requirements.txt
```

> First install downloads ~80 MB embedding model (`all-MiniLM-L6-v2`) on first run and caches it.

🔑 Environment Variable Setup

Local development

1. Copy `.env.example` → `.env`:

```
copy .env.example .env    # Windows
# cp .env.example .env    # macOS/Linux
```

2. Edit `.env`:

```
GROQ_API_KEY=gsk_your_actual_key_here
GROQ_MODEL=openai/gpt-oss-120b
```

- `GROQ_API_KEY` — **required**. Get at https://console.groq.com/keys
- `GROQ_MODEL` — **configurable**. Use a currently available model (verified 2026-08-29):
  - `openai/gpt-oss-120b` (recommended, fast, default)
  - `openai/gpt-oss-20b` (smaller, faster)
  - `allam-2-7b`
  - `groq/compound` / `groq/compound-mini`
  - `qwen/qwen3-32b`
  - See https://console.groq.com/docs/models for the current list
  - ⚠ `llama-3.1-8b-instant` / `mixtral-8x7b-32768` / `gemma2-9b-it` are now 404/decommissioned

> If the model is decommissioned, the app shows: "Groq model 'X' unavailable... set GROQ_MODEL in .env and restart".

Streamlit Community Cloud (deployment)

- Do **not** upload `.env`.
- In Streamlit dashboard: **Settings** → **Secrets** → add (see `.streamlit/secrets.toml.example`):

```
GROQ_API_KEY = "gsk_..."
GROQ_MODEL = "openai/gpt-oss-120b"
```

The app also reads `st.secrets` if `.env` is not present (via `os.getenv`).

▶ How to Run

```
# Activate venv if not already
venv\Scripts\activate        # Windows
# source venv/bin/activate   # macOS/Linux

streamlit run app.py
```

Open the URL shown (typically http://localhost:8501).

Alternative (Flask backup):

```
python app_flask_backup.py   # original Flask app on http://localhost:5000
```

How RAG Works (in this project)

1. **Ingest:** User uploads PDF(s). PyMuPDF extracts text **per page**, preserving `filename` + `page_number`. Scanned PDFs (no text) trigger a warning.
2. **Chunk:** `RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100, separators=["\n\n","\n"," ",""])` — sentence-aware, overlap preserves facts that span chunks.
3. **Embed:** `SentenceTransformer("all-MiniLM-L6-v2")` converts each chunk to a **384-dim float32** vector. Batch size 32, model cached with `@st.cache_resource` (loads once).
4. **Store:** Vectors added to **FAISS** `IndexFlatL2` (`faiss.IndexFlatL2(dim)`). `np.ascontiguousarray(float32)`. Index + chunks stored in `st.session_state` so questions don't rebuild it.
5. **Retrieve:** Question is embedded with same model → `index.search(q_emb, top_k)` → top 3–5 chunks (clamped if fewer chunks). L2 distance converted to similarity for display.
6. **Generate:** Context = joined retrieved chunks with `Source: filename | Page N` tags. System prompt: *"Answer only from context. If not in context, say 'The answer is not available…' Do not invent."* Sent to **Groq** `chat.completions.create(model=GROQ_MODEL, temperature=0.2)`. Strips `<think>` tags.
7. **Cite:** Sources deduplicated by `(filename, page_number)` and shown as badges. Expanders show raw retrieved chunks for audit.

Project Workflow (User Perspective)

1. Open app → see title, description, architecture expander
2. Upload PDF(s) → see file names + "Process PDFs & Build Vector Index" button
3. Click Process → spinner: extracting → embedding (batch) → FAISS → toast + success → stats update (PDFs, chunks, vectors, dim)
4. Enter question → click Ask → spinner: retrieval → Groq → answer appears with sources + chat history
5. Expand "View retrieved chunks" to audit context
6. Ask follow-ups → history appends (session-based)
7. Clear Chat → history cleared, index retained (or Clear All → index + history cleared)

Testing

See **TESTING.md** for 12 manual test cases:

- Single / multiple PDF upload, duplicate, large PDF
- Valid question (answer present / absent)
- Empty / scanned / invalid PDF
- Empty question, API failure, multiple questions, chat history

Quick sanity check after `pip install`:

```
python -c "import fitz, faiss, sentence_transformers, streamlit, groq;
print('imports OK')"
python -m py_compile app.py && echo "syntax OK"
streamlit run app.py
# Upload a PDF, ask: "What is RAG?" → expect grounded answer + 📄 file | Page
N
# Ask nonsense: "Who won the 3025 World Cup?" → expect "I couldn't find this
information in the uploaded documents."
```

☁️ Deployment (Streamlit Community Cloud)

1. Push to GitHub **without** `.env` (`.gitignore` already excludes it):

```
git add app.py utils/ requirements.txt .gitignore .env.example README.md
TESTING.md
git commit -m "feat: Streamlit RAG with FAISS+Groq"
git push
```

2. On https://share.streamlit.io → **New app** → select repo → branch `main` → main file `app.py`

3. **Advanced** → **Secrets** → add (app supports `st.secrets` with fallback to `os.getenv`):

```
GROQ_API_KEY = "gsk_..."
GROQ_MODEL = "openai/gpt-oss-120b"
```

> Locally the app uses `.env` (`python-dotenv`); on Cloud it automatically reads `st.secrets["GROQ_API_KEY"]` then falls back to env — no code change needed.

4. Deploy. No Windows paths are hard-coded; `requirements.txt` is pinned; app works from clean install. Test with: `streamlit run app.py`

🔮 Future Enhancements

- OCR for scanned PDFs (Tesseract / pytesseract) + image extraction
- Multi-format support (DOCX, TXT, PPTX, CSV) and URL ingestion
- Chunking by tokens (tiktoken) + semantic splitter + reranking (cross-encoder)
- Hybrid search (BM25 + FAISS) and metadata filtering (date, author)
- Conversation memory (multi-turn QA with history-aware retrieval)
- User auth, per-user collections, persistent cloud vector DB (Chroma / Qdrant / Pinecone)
- Evaluation dashboard (RAGAS: faithfulness, relevance, recall) + feedback loop
- Streaming answers and voice Q&A

⚠ Limitations

- Scanned/image-only PDFs need OCR (not included; shows warning instead)
- `all-MiniLM-L6-v2` is lightweight (384-dim) — domain jargon may need larger model
- FAISS `IndexFlatL2` is exact search — slows beyond ~100k chunks (use IVF/PQ for scale)
- Context window limited to top-k chunks (max 10) — very long answers may be truncated (900 tokens)
- Groq free tier has rate limits; large PDFs may hit token limits
- No persistent storage across sessions (in-memory; add vector DB persistence for prod)

📸 Screenshots (placeholders)

Add screenshots after running:

- `screenshots/upload.png` — PDF upload + file names
- `screenshots/processing.png` — spinner + stats (PDFs, chunks, vectors, dim)
- `screenshots/question.png` — question input + Ask button
- `screenshots/answer.png` — answer + source badges + expanders
- `screenshots/chat_history.png` — multiple Q/A + Clear Chat

Create a `screenshots/` folder and embed:

```
![Upload](screenshots/upload.png)
![Answer with Sources](screenshots/answer.png)
```

👤 Author

B.Tech Major Project — AI-Powered Document QA using RAG

- Student: *[Your Name]*
- Guide: *[Guide Name]*
- Department: *[CSE / AI&DS]*
- Year: 2025–2026

Credits: LangChain, Sentence Transformers, FAISS, Groq, Streamlit, PyMuPDF.

📄 License

MIT — for academic use. Keep `GROQ_API_KEY` private.